

Detection of Unsafe Hanging Travel in Public Transport Using Machine Learning

Tazkia Malik, Sadek Hossain Asif, Tanzila Khan
Taif Ahmed Turjo, Amio Rashid, Mahmud Hasan Walid
Department of Computer Science and Engineering
University of Dhaka
Dhaka, Bangladesh

Abstract

Unsafe door-hanging and overcrowded travel are persistent safety risks in Dhaka’s public transport system, particularly in buses and legunas operating under high passenger demand. Existing intelligent transportation research has addressed vehicle detection, passenger counting, and general crowd analysis, but localized automated detection of unsafe hanging travel remains limited. This paper presents an image-based machine-learning framework for classifying public transport scenes into safe/normal and unsafe/risky conditions. A custom bus and leguna image dataset is prepared through cleaning, annotation, preprocessing, and augmentation. For classical machine-learning baselines, Histogram of Oriented Gradients (HOG) descriptors are combined with Logistic Regression, Gaussian Naive Bayes, and a Support Vector Machine (SVM) using a radial basis function kernel. Deep models including a CNN trained from scratch, ResNet18, EfficientNet-B0, ResNet50, ConvNeXt-Tiny, and an ensemble are also evaluated to compare learned visual features with handcrafted HOG representations. The retrained dataset contains 523 annotation-labeled images, and augmentation uses train-only augmentation that avoids augmentation leakage. On the untouched 79-image test set, the ResNet50–ConvNeXt ensemble reaches 94.7% unsafe recall, 86.1% accuracy, and 0.970 ROC-AUC at the balanced operating point, while ResNet50 attains the highest single-model accuracy of 92.4%. Because this is intended for safety-oriented monitoring, the system is tuned to prioritize unsafe-class recall while retaining acceptable discrimination. The current run yields promising unsafe-hanging-travel classification performance, while the small test set limits the precision of the estimate.

Keywords: Unsafe public transport, computer vision, image classification, Histogram of Oriented Gradients, support vector machine, convolutional neural network, intelligent transportation system.

1 Introduction

Public transport in Dhaka carries a large share of daily urban mobility, but high passenger demand, limited vehicle capacity, and inconsistent enforcement often create unsafe conditions. A common visible hazard is door-hanging travel, where passengers ride from the doorway or exterior boundary of buses and legunas, increasing exposure to falls, collisions, and roadside obstacles.

Safety monitoring still depends heavily on traffic personnel, transport operators, or surveillance staff. Manual observation is difficult to scale across dense road networks, heterogeneous vehicle types, and continuously changing traffic scenes.

Computer vision is useful for this task because images can capture vehicle structure, passenger position, crowding, and doorway conditions. This paper proposes an image-based machine-learning system that classifies Dhaka public transport scenes into safe/normal negative samples and unsafe/risky positive samples.

The main contributions are as follows:

- A localized safe/unsafe public transport image dataset containing Dhaka bus and leguna scenes.
- A complete machine-learning pipeline for unsafe hanging-travel detection.
- A comparison of HOG-based classical machine-learning models and deep convolutional models.
- An evaluation using standard classification metrics, including accuracy, unsafe-class recall, ROC and precision–recall curves, and AUC, together with a train/validation/test generalization analysis.

2 Related Work

Computer vision has become an important component of intelligent transportation systems. Surveyed applications include vehicle detection, pedestrian detection, anomaly detection, traffic monitoring, and automatic number-plate recognition [1]. These methods show that visual data can support automated interpretation of transportation scenes, although most work targets general traffic understanding rather than specific passenger-safety risks.

Passenger monitoring research commonly focuses on counting, occupancy estimation, boarding analysis, and crowd monitoring in buses or terminals [2]. Public transport safety studies identify overcrowding, weak enforcement, and unsafe passenger behavior as important risk factors in dense urban environments [3], while broader reviews show that crowd pressure and non-collision incidents remain persistent concerns when demand exceeds service capacity [4].

Unsafe behavior and abnormal-activity detection are related to the present task because they aim to identify visual patterns that deviate from normal scenes. Recent surveys report that deep learning can be effective for abnormal

behavior detection, but they also emphasize difficulties caused by limited labeled data, occlusion, scene variation, and domain transfer [5]. Object-detection methods such as YOLO provide another foundation for transportation surveillance and boarding or alighting recognition [6], [7]. Closer to the present task, vision-based systems have been developed to recognize abnormal or unsafe passenger behavior inside buses from onboard cameras [8], [9], [10], and to analyze passenger posture during the safety-critical boarding and alighting phase at bus doors [11]. These efforts confirm that onboard and door-region imagery carries strong cues for passenger-safety assessment, but they target in-vehicle activity or door movement rather than passengers hanging from the exterior of a moving vehicle.

Automatic License Plate Recognition (ALPR) is relevant only as future context for identifying vehicles after unsafe behavior is detected. Existing ALPR studies address plate localization and recognition under real-world imaging constraints [12], [13], [14], including Bengali license-plate recognition under low-resolution and blurred conditions [15].

The key gap is that existing work rarely addresses unsafe door-hanging detection in Dhaka-like crowded public transport environments using localized datasets under real-world challenges such as occlusion, motion blur, lighting variation, crowding, and non-standard vehicle structures. Prior systems often estimate occupancy, count passengers, or recognize boarding events, but they do not directly model hanging passengers as a safety-critical positive class under local bus and leguna conditions. Safety interventions for footboard travel in South Asian buses have relied mainly on onboard IoT sensors rather than vision [17], and although localized Bangladeshi image datasets exist for vehicle-type classification [16], none label unsafe door-hanging behavior.

3 Dataset

3.1 Data Collection

The dataset contains images of buses and legunas captured under real-world public transport conditions in Dhaka. The samples cover safe/normal and unsafe/risky cases with variation in vehicle type, passenger density, camera angle, lighting, and scene clutter. Labels are derived from image annotations: an image is unsafe if it contains at least one box labeled **unsafe**; otherwise it is safe, and license-plate boxes are ignored. The retrained dataset contains 523 labeled originals, with 272 safe samples and 251 unsafe samples. Figure 1 summarizes the updated class distribution and split composition, and Table 1 lists the per-partition counts.

Representative examples of the vehicle categories are shown in Figure 2 and Figure 3.

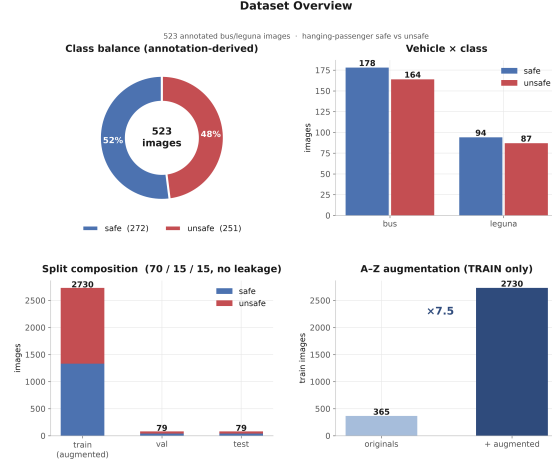


Figure 1: Dataset overview: annotation-derived class balance, vehicle-wise class distribution across buses and legunas, stratified 70/15/15 split composition, and train-only augmentation growth from 365 originals to 2,730 training images.

Table 1: Dataset Composition After Splitting and Train-Only Augmentation

Partition	Total	Safe	Unsafe
Training (augmented)	2,730	1,330	1,400
Validation	79	41	38
Test	79	41	38
Labeled originals	523	272	251



(a) Negative class.



(b) Positive class.

Figure 2: Representative leguna samples.



(a) Negative class.



(b) Positive class.

Figure 3: Representative bus samples.

3.2 Data Cleaning

Cleaning removed images whose content was blurred, ambiguous, low quality, heavily obstructed, poorly framed, or unreliable for class assignment. This step reduced label noise while preserving realistic variation in traffic, vehicle structure, crowding, and illumination.

3.3 Data Annotation

Manual annotation assigned each retained image to one of two classes: negative for safe/normal passenger conditions and positive for unsafe/risky hanging travel. Figure 4 shows representative annotation examples.

3.4 Data Augmentation

The source images were EXIF-orientation-corrected, converted to RGB, resized so the shorter side was 512 pixels, center-cropped to 512×512 , and written as PNG before splitting. The repository stores raw images and annotations in Cloudflare R2; the available records do not document the original acquisition procedure or collection period. Only the 365 training originals were augmented offline, producing 2,365 copies and 2,730 training images with 1,330 safe and



(a) Unsafe/risky annotation.



(b) Safe/normal annotation.

Figure 4: Representative annotation examples.

1,400 unsafe samples; validation and test remain 79 unaugmented originals each (41 safe and 38 unsafe). Offline augmentation used the implemented Albumentations pipeline: horizontal flip ($p = 0.50$); one of affine transformation (scale 0.90–1.12, translation up to 8%, rotation $\pm 15^\circ$, shear $\pm 8^\circ$), perspective distortion (scale 0.02–0.08), or random-resized crop (scale 0.78–1.00, aspect ratio 0.8–1.25) with $p = 0.90$; brightness/contrast, gamma (70–140), or color jitter with $p = 0.85$; hue-saturation-value, RGB shift, or CLAHE with $p = 0.55$; sharpening or embossing ($p = 0.20$); blur ($p = 0.30$); noise ($p = 0.30$); compression or downscaling ($p = 0.25$); shadow, fog, or sun flare ($p = 0.15$); coarse dropout ($p = 0.30$); and grayscale ($p = 0.05$). Thus, offline transforms are applied after splitting and only to training originals. The public implementation gives the exhaustive transform parameterization.

During deep-model training, the online pipeline applies random-resized crop (scale 0.8–1.0, ratio 0.8–1.25), horizontal flip, $\pm 10^\circ$ rotation, RandAugment (two operations, magnitude 7), ColorJitter (0.25, 0.25, 0.25, 0.05), tensor conversion, ImageNet normalization, and random erasing ($p = 0.25$, scale 0.02–0.12). Evaluation preprocessing uses resize to 1.15 times the model input followed by center crop and ImageNet normalization. Test-time augmentation averages the original and horizontally flipped evaluation predictions for deep models. Weighted random sampling and class-weighted cross-entropy or focal loss are training controls, not image transforms. These transformations simulate common roadside imaging variation while preserving the class label [18].

All images were standardized before model-specific processing. The standardized preprocessing and model input settings are summarized in Table 2. The dataset is available through Mendeley Data.

3.5 Ethical Considerations

The dataset is used only for academic safety-monitoring research. Images may contain privacy-sensitive visual information such as human faces, vehicle license plates, and location-related context. The classification task focuses on transport safety conditions rather than personal identity recognition; no face recognition, identity matching, or individual profiling is performed. Before any public release or operational deployment, privacy-preserving steps such as face and license-plate anonymization, removal of GPS or location metadata, consent and data-governance procedures, and compliance with relevant regulations would be required.

4 Methodology

The proposed method is a binary image-classification pipeline for public transport safety assessment. The workflow consists of dataset preparation, preprocessing, feature extraction or model-specific input preparation, model training, and evaluation. HOG-based classical models receive handcrafted gradient features, while deep neural models learn directly from image tensors. Implementation settings are summarized in Table 2.

4.1 Pipeline Overview

The pipeline begins with Dhaka bus and leguna image collection, followed by cleaning, annotation, augmentation, and standardization. Classical machine-learning models use HOG feature vectors extracted from grayscale inputs, whereas CNN and transfer-learning models receive RGB tensors. All models are evaluated on the same held-out split using standard classification metrics.

4.2 Training and Inference Procedure

The complete retraining procedure is summarized below so that data preparation, model fitting, and deployment decisions are defined within the methodology.

1. **Annotation-based labeling:** Begin with 523 labeled originals containing 272 safe and 251 unsafe images. Assign an image to the unsafe class when at least one annotation box is labeled **unsafe**; ignore license-plate boxes.
2. **Four-way stratification:** Split the originals 70/15/15 over vehicle type and class, preserving bus-safe, bus-unsafe, leguna-safe, and leguna-unsafe samples in validation and test partitions. The code does not perform scene-ID, event, or near-duplicate grouping, so this stratification alone does not establish independence at those levels.

3. **Train-only augmentation:** Apply offline augmentation only to the 365 training originals, avoiding augmentation leakage and producing 2,730 training images. Keep the 79-image validation and 79-image test partitions unaugmented.
4. **Model-specific preparation:** Extract standardized HOG features followed by PCA for classical models. Supply normalized RGB tensors to the scratch CNN and ImageNet-pretrained transfer-learning models.
5. **Model training:** Train Logistic Regression, Gaussian Naive Bayes, SVM, and the scratch CNN as baselines. For each pretrained backbone, train the new head alone for the configured warm-up, then unfreeze the entire backbone and continue fine-tuning with discriminative learning rates. Use weighted sampling and the configured weighted cross-entropy or focal loss.
6. **Validation and ensembling:** Tune balanced, high-recall (at least 95% validation unsafe recall), and max-recall (100% validation unsafe recall) thresholds on validation predictions. The final model-selection procedure fixes the final ensemble as the probability average of ResNet50 and ConvNeXt-Tiny; this choice and all thresholds are made before test evaluation.
7. **Test-time inference:** Apply horizontal-flip test-time augmentation to the deep models, average the resulting probabilities, select the configured operating threshold, and evaluate once on the untouched test set.

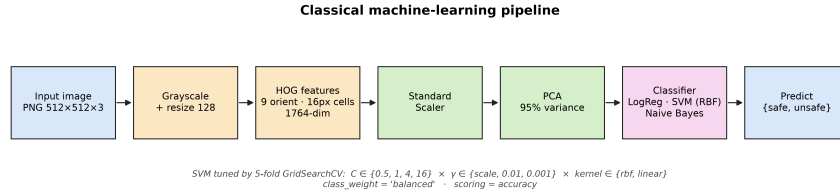


Figure 5: Overview of the HOG-based classical machine-learning pipeline.

4.3 HOG-Based Feature Representation

For the classical models, Histogram of Oriented Gradients (HOG) descriptors were extracted from grayscale images [19]. HOG captures local gradient orientation and edge-structure information, which is useful for posture-, shape-, and boundary-related visual classification. Feature extraction was implemented with scikit-image [20]; Table 2 lists the extraction settings, and Figure 5 shows the full classical pipeline.

4.4 Classical Machine Learning Models

The classical baselines use HOG features with three complementary classifiers. Logistic Regression provides a linear probabilistic baseline and estimates the posterior probability of the positive (unsafe) class as

$$P(y = 1 \mid \mathbf{x}) = \sigma(\mathbf{w}^\top \mathbf{x} + b) = \frac{1}{1 + \exp(-(\mathbf{w}^\top \mathbf{x} + b))}, \quad (1)$$

where $\mathbf{x} \in \mathbb{R}^d$ is the PCA-reduced HOG feature vector, $\mathbf{w} \in \mathbb{R}^d$ and $b \in \mathbb{R}$ are the learned weight vector and bias, and $\sigma(\cdot)$ is the logistic sigmoid. The model is computationally efficient and interpretable, but its linear decision boundary can be limited when safe and unsafe samples are not linearly separable.

Gaussian Naive Bayes is a probabilistic baseline that assumes conditional independence among features given the class label. Each feature x_i is modeled with a class-conditional Gaussian likelihood,

$$\begin{aligned} p(x_i \mid y) &= \mathcal{N}(x_i; \mu_{y,i}, \sigma_{y,i}^2) \\ &= \frac{1}{\sqrt{2\pi} \sigma_{y,i}} \exp\left(-\frac{(x_i - \mu_{y,i})^2}{2\sigma_{y,i}^2}\right), \end{aligned} \quad (2)$$

where $\mu_{y,i}$ and $\sigma_{y,i}^2$ are the mean and variance of feature i under class y . Assuming conditional independence, the predicted label maximizes the class posterior,

$$\hat{y} = \arg \max_{y \in \{0,1\}} P(y) \prod_{i=1}^d p(x_i \mid y). \quad (3)$$

Although the independence assumption is restrictive for correlated HOG features, the model is useful as a low-complexity comparator. The Support Vector Machine (SVM) identifies a decision boundary that maximizes the margin between the negative and positive classes [21]. To capture non-linear patterns in the HOG feature space, the radial basis function (RBF) kernel is used to quantify the similarity between pairs of feature vectors,

$$K(\mathbf{x}_i, \mathbf{x}_j) = \exp(-\gamma \|\mathbf{x}_i - \mathbf{x}_j\|_2^2), \quad \gamma > 0, \quad (4)$$

and the decision function for a test vector $\mathbf{x}$ is

$$f(\mathbf{x}) = \text{sgn}\left(\sum_{i \in \mathcal{S}} \alpha_i y_i K(\mathbf{x}_i, \mathbf{x}) + b\right), \quad (5)$$

where $\mathcal{S}$ indexes the support vectors, $\alpha_i \geq 0$ are the learned dual coefficients, $y_i \in \{-1, +1\}$ are the training labels, and b is the bias. The penalty C , kernel width γ , and kernel type were selected by five-fold grid search on the training data. Table 2 summarizes the implementation settings for all three classical models.

4.5 Deep Neural Models

The deep models process RGB images directly and learn spatial features from the training data [22]. The experiments evaluate five neural architectures: a CNN trained from scratch, ResNet18, EfficientNet-B0, ResNet50, and ConvNeXt-Tiny. The scratch CNN provides a lightweight learned-feature baseline, while the other architectures use ImageNet-pretrained weights and two-phase fine-tuning [23], [24], [25]. A probability-level ensemble combines ResNet50 and ConvNeXt-Tiny. The scratch CNN uses batch normalization [26]; dropout is applied where specified in the training configurations [27]. Weighted sampling, weighted loss, validation-tuned thresholds, and horizontal-flip test-time augmentation address imbalance and improve unsafe-class recall. Figure 6 shows the scratch CNN architecture and Figure 7 illustrates the two-phase transfer-learning scheme shared by the pretrained models; Table 2 reports the combined configuration.

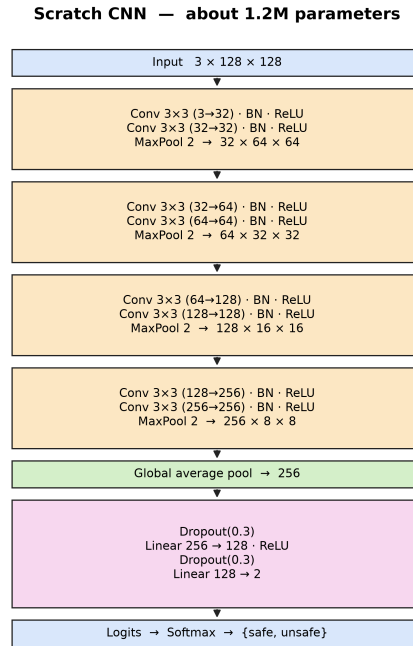


Figure 6: Scratch CNN architecture used for image-level binary classification.

Two-phase transfer learning (ResNet18 shown) — about 11M parameters, ImageNet-pretrained

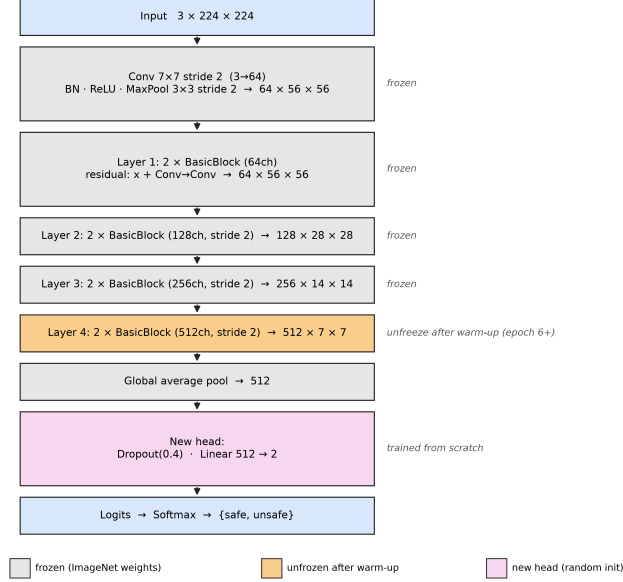


Figure 7: Two-phase transfer-learning scheme, illustrated with ResNet18. The ImageNet-pretrained backbone is frozen during head-only warm-up; after warm-up, all backbone parameters are unfrozen and fine-tuned with a lower learning rate than the new classifier head. ResNet50, EfficientNet-B0, and ConvNeXt-Tiny follow the same procedure at a 224×224 input resolution.

5 Experimental Setup

5.1 Split Protocol

The data split followed a four-way stratified protocol over vehicle type and class, producing 70/15/15 training, validation, and test partitions. This ensures that bus-safe, bus-unsafe, leguna-safe, and leguna-unsafe samples occur in both validation and test sets. Augmentation was applied only after splitting and only to training originals. Thresholds were tuned on validation predictions, while the 79-image test set remained untouched until final evaluation. The final model-selection procedure fixes ResNet50 and ConvNeXt-Tiny as ensemble members and derives the balanced, high-recall, and max-recall thresholds from validation predictions before test scoring; the test set was not used for threshold tuning or ensemble construction. The full nine-model test table is reported for transparency.

5.2 Model Configuration

The experiments were implemented in Python using scikit-learn for the classical pipelines [29], PyTorch for the neural model [30], and scikit-image for image processing [20]. The training script records the seeds for each candidate run; model and preprocessing settings are consolidated in Table 2.

Table 2: Model Configuration

Model / Stage	Input Representation	Key Configuration
Image standardization	Source images	Orientation correction, RGB conversion, shorter side 512 px, center crop to 512×512 , PNG output.
HOG feature extraction	Grayscale 128×128	9 orientation bins, 16×16 pixel cells, 2×2 cell blocks, L2-Hys normalization, 1764-d feature vector.
Logistic Regression	HOG features	StandardScaler and PCA (95% variance retained) on training data, then Logistic Regression (lbfgs , balanced class weights).
Gaussian Naive Bayes	HOG features	StandardScaler and PCA (95% variance retained) on training data, then Gaussian Naive Bayes.
SVM with RBF kernel	HOG features	StandardScaler and PCA (95% variance retained) on training data; RBF-kernel SVM with penalty C , kernel width γ , and kernel selected by 5-fold GridSearchCV (balanced class weights, probability estimates enabled).
CNN (from scratch)	RGB 128×128	Four convolutional blocks (32/64/128/256 channels) with batch normalization and ReLU, global average pooling, dropout, and a two-layer linear head; about 1.2M parameters.
ResNet18	RGB 224×224	ImageNet V1; two-phase full-backbone fine-tuning; hflip TTA.
EfficientNet-B0	RGB 224×224	ImageNet V1; two-phase full-backbone fine-tuning; hflip TTA.
ResNet50	RGB 224×224	ImageNet V2; two-phase full-backbone fine-tuning; hflip TTA.
ConvNeXt-Tiny	RGB 224×224	ImageNet V1; two-phase full-backbone fine-tuning; hflip TTA.
Ensemble	RGB 224×224	Probability-level average of ResNet50 and ConvNeXt-Tiny, with balanced, high-recall, and max-recall thresholds tuned on validation.
Deep-model training	RGB tensors	Common protocol and recorded candidate-run settings are documented below; the checkpoint-selection rule is highest validation balanced accuracy, but the exact winning configuration is not recoverable from the saved artifacts.
Evaluation protocol	Train/ validation/test	Four-way stratified 70/15/15 split; 365 training originals expanded to 2,730 train images, with 79 unaugmented validation and 79 untouched test images.

The final protocol used ImageNet-pretrained transfer backbones with a head-only warm-up followed by full-backbone fine-tuning, AdamW, cosine annealing,

weighted sampling, horizontal-flip TTA, and checkpoint selection by validation balanced accuracy. The training records document candidate configurations, including batch sizes of 24 or 32, 45–70 total epochs, 5 or 6 warm-up epochs, model-specific learning rates and weight decay, dropout, label smoothing, and focal-loss alternatives.

For HOG-based models, preprocessing and dimensionality-reduction steps were fitted on the training data and then applied to validation and test data. Deep models used normalized RGB image tensors rather than HOG descriptors.

For neural-model inference, the output logits $\mathbf{z} = (z_0, z_1) \in \mathbb{R}^2$ are mapped to class probabilities by the softmax function,

$$p_k = \frac{\exp(z_k)}{\sum_{j=0}^1 \exp(z_j)}, \quad k \in \{0, 1\}, \quad (6)$$

so that p_1 is the estimated probability of the unsafe class. Neural models were trained with the AdamW optimizer, a decoupled weight-decay variant of Adam [31], under a cosine-annealing learning-rate schedule; the complete training configuration is listed in Table 2.

5.3 Evaluation Metrics

Model performance was assessed using accuracy, balanced accuracy, precision, recall, specificity, the F_1 -score, the Matthews correlation coefficient (MCC), ROC-AUC, and PR-AUC. Since the two classes are approximately balanced, both ROC-AUC and PR-AUC are reported to provide a more complete view of classification performance. Particular attention is given to recall for the unsafe/risky class, as failing to identify an unsafe case carries greater practical concern. Let TP , TN , FP , and FN denote the numbers of true positives, true negatives, false positives, and false negatives, respectively, where unsafe/risky hanging travel is treated as the positive class. The threshold-dependent evaluation metrics are defined as follows:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}, \quad (7)$$

$$\text{Precision} = \frac{TP}{TP + FP}, \quad \text{Recall} = \frac{TP}{TP + FN}, \quad (8)$$

$$\text{Specificity} = \frac{TN}{TN + FP}, \quad (9)$$

$$F_1 = \frac{2 \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} = \frac{2TP}{2TP + FP + FN}. \quad (10)$$

Recall (equivalently, sensitivity or the true positive rate) quantifies unsafe-class coverage, whereas specificity (the true negative rate) quantifies safe-class coverage. Because a single decision threshold can favor one class, balanced accuracy

reports the unweighted mean of the two per-class recalls and is therefore insensitive to class prevalence,

$$\begin{aligned} \text{Balanced Accuracy} &= \frac{\text{Recall} + \text{Specificity}}{2} \\ &= \frac{1}{2} \left(\frac{TP}{TP + FN} + \frac{TN}{TN + FP} \right). \end{aligned} \quad (11)$$

The Matthews correlation coefficient (MCC) measures the correlation between the predicted and true binary labels. By incorporating all four entries of the confusion matrix, it produces a high score only when the classifier performs consistently well across both classes. This makes MCC a particularly informative single-value measure, especially when class distributions are imbalanced [28]:

$$\text{MCC} = \frac{TP \cdot TN - FP \cdot FN}{\sqrt{(TP + FP)(TP + FN)(TN + FP)(TN + FN)}}. \quad (12)$$

The coefficient ranges from -1 to $+1$. A value of $+1$ represents perfect agreement between predictions and true labels, 0 indicates no correlation, and -1 reflects complete disagreement. When the denominator is undefined because one of its factors is zero, the coefficient is conventionally assigned a value of 0 . In addition to these threshold-dependent measures, two threshold-independent metrics are used to assess ranking performance across all possible decision thresholds. ROC-AUC measures the area under the receiver operating characteristic curve, which plots the true positive rate against the false positive rate. It can also be interpreted as the probability that the model assigns a higher score to a randomly selected unsafe image than to a randomly selected safe image. PR-AUC, reported as average precision, summarizes the relationship between precision and recall across different classification thresholds.

$$\text{AP} = \sum_n (R_n - R_{n-1}) P_n, \quad (13)$$

where P_n and R_n are the precision and recall at the n -th threshold; its no-skill baseline equals the unsafe-class prevalence, which makes it informative for the safety-critical positive class. Because missed unsafe cases have direct safety implications, unsafe-class recall is emphasized alongside overall accuracy, MCC, and AUC. Given the small test set, uncertainty is reported as 95% Wilson score intervals, and pairwise differences between the strongest models are assessed with McNemar’s exact test. For the ensemble, unsafe recall is $36/38 = 94.7\%$, with a 95% Wilson interval of $[82.7\%, 98.5\%]$.

6 Results

6.1 Quantitative Comparison

Table 3 reports balanced operating-point performance on the untouched 79-image test set. The ResNet50-ConvNeXt ensemble reaches 0.861 accuracy,

0.947 unsafe recall, and 0.970 ROC-AUC. ResNet50 attains the highest single-model accuracy (0.924) and MCC (0.848), while EfficientNet-B0 attains the highest unsafe recall (0.974) and ROC-AUC (0.979). All nine models are compared in Figure 8.

Table 3: Held-out Test Performance for All Reported Models

Model	Acc.	Bal. Acc.	Unsafe R	Unsafe P	Spec.	F_1	AUC	PR-AUC	MCC
Ensemble	0.861	0.864	0.947	0.800	0.780	0.867	0.970	0.978	0.734
ResNet50	0.924	0.924	0.921	0.921	0.927	0.921	0.967	0.975	0.848
ConvNeXt-Tiny	0.835	0.840	0.947	0.766	0.732	0.847	0.972	0.979	0.691
EfficientNet-B0	0.835	0.841	0.974	0.755	0.707	0.851	0.979	0.980	0.701
ResNet18	0.911	0.912	0.921	0.897	0.902	0.909	0.971	0.976	0.823
CNN	0.861	0.861	0.868	0.846	0.854	0.857	0.929	0.896	0.722
SVM (RBF)	0.873	0.874	0.895	0.850	0.854	0.872	0.952	0.958	0.748
Logistic Regression	0.810	0.810	0.816	0.795	0.805	0.805	0.887	0.900	0.620
Naive Bayes	0.734	0.734	0.737	0.718	0.732	0.727	0.866	0.884	0.468

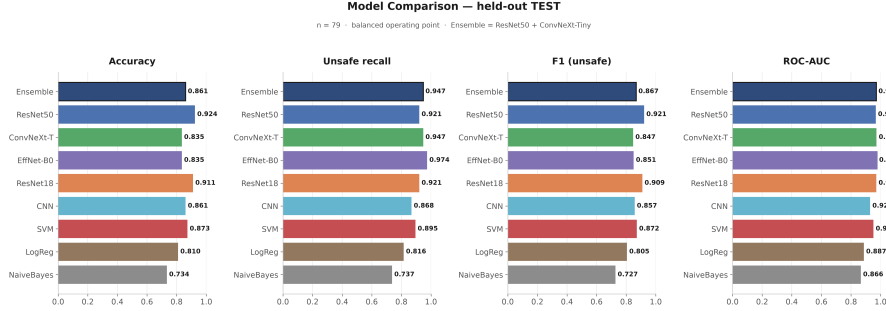


Figure 8: Held-out test comparison of accuracy, unsafe recall, unsafe F_1 , and ROC-AUC across all nine reported models.

6.2 ROC and AUC Analysis

AUC values are reported in Table 3. Figure 9a shows held-out ROC curves for all reported models, and Figure 9b shows the corresponding precision–recall curves.

6.3 Error Analysis

False negatives are the most safety-critical error type because they correspond to unsafe/risky hanging-travel cases predicted as safe/normal. At the balanced operating point, the ensemble detects 94.7% of unsafe test images, missing only 2 of 38 unsafe test images. The balanced, high-recall, and max-recall thresholds are respectively 0.170, 0.175, and 0.175, all tuned on validation; they happen to produce the same rounded test result. Table 4 lists the three modes, and Figure 10 shows the threshold trade-off.

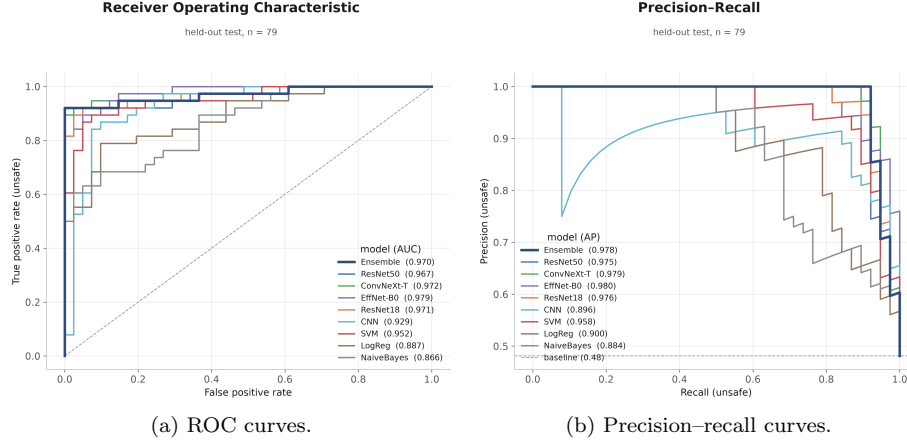


Figure 9: Receiver operating characteristic and precision-recall curves comparing all nine models on the held-out test set.

Table 4: Selectable operating points for the deployed ensemble. All thresholds are tuned on validation; the displayed test metrics are evaluated afterward.

Mode	Threshold	Test Acc.	Unsafe R	Unsafe P	Spec.
Balanced	0.170	0.861	0.947	0.800	0.780
High recall	0.175	0.861	0.947	0.800	0.780
Max recall	0.175	0.861	0.947	0.800	0.780

7 Discussion

7.1 Interpretation of Results

The retrained ensemble provides a strong, recall-oriented balanced operating point, reaching 86.1% test accuracy, 94.7% unsafe recall, 0.867 unsafe F_1 , 0.970 ROC-AUC, and 0.978 PR-AUC. The current run shows strong ranking performance, while the small test set warrants cautious interpretation of the estimate.

Among individual deep models, ResNet50 leads on accuracy (92.4%) with balanced 92.1% unsafe precision and recall, while EfficientNet-B0 reaches the highest unsafe recall (97.4%) and the top ROC-AUC (0.979). Among classical models, SVM remains the strongest HOG baseline, reaching 87.3% accuracy, 89.5% unsafe recall, and 0.952 AUC.

The deployed configuration uses the ResNet50-ConvNeXt ensemble because the final model-selection procedure fixes these two architecturally different transfer backbones before test evaluation and averages their unsafe-class probabilities. This is consistent with the safety-oriented objective of prioritizing unsafe recall while retaining acceptable discrimination. On the test set, the ensemble reaches 94.7% unsafe recall, compared with ResNet50’s 92.1%;

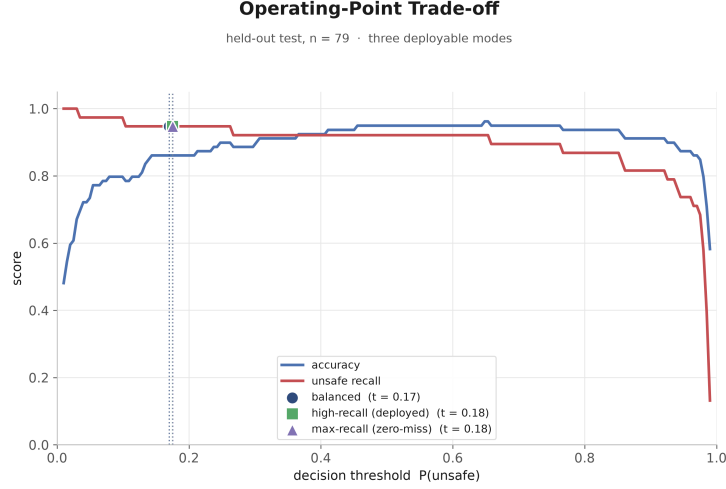


Figure 10: Ensemble threshold behavior on the held-out test set: accuracy and unsafe recall against the decision threshold, with the operating points selected from validation predictions marked. Because the retrained dataset is near-balanced, the three points nearly coincide.

EfficientNet-B0 reaches 97.4% unsafe recall but has lower balanced accuracy (83.5%). These test-set comparisons are descriptive and were not used to select the ensemble.

Gaussian Naive Bayes underperforms because its conditional-independence assumption is poorly matched to HOG descriptors. Neighboring gradient bins and block-normalized features are correlated by construction, so treating them as independent weakens the model’s ability to distinguish visually similar safe/normal and unsafe/risky scenes.

The train-to-test gaps in Table 5 range from 6.7 to 16.5 percentage points. Logistic Regression has the smallest gap, while ConvNeXt-Tiny has the largest. These current-run gaps show that performance differs between the training and held-out partitions; they do not, by themselves, support a before-and-after claim about reduced overfitting. Figure 11 visualizes the per-model train, validation, and test error and the train-to-test gap.

For this application, false negatives are more consequential than ordinary classification errors because they correspond to missed risky travel conditions. Precision and overall accuracy remain relevant for reducing false alarms, but a safety-oriented monitoring system should prioritize reducing missed unsafe cases while maintaining acceptable discrimination across normal and risky scenes.

7.2 Limitations

The main limitation of this study is the limited number of unique source images. The updated dataset contains 523 labeled originals with 251 unsafe cases,

which is still not enough to fully cover the diversity of vehicle layouts, camera viewpoints, passenger postures, weather, illumination, and traffic environments encountered in deployment. With only 79 test images, individual metrics carry wide uncertainty: a 95% Wilson score interval on the ensemble’s 86.1% accuracy spans $[76.8, 92.0]\%$. A McNemar exact test finds no significant difference among the strongest models (for example, ResNet50 versus the ensemble gives $p = 0.125$ and ResNet50 versus ResNet18 gives $p = 1.0$), so these models are statistically indistinguishable on this test set, and the per-metric maxima in Table 3 should be read as descriptive rather than as proof of a single definitively best model. Because nine models are compared on one split, the top scores are also subject to selection (“winner’s-curse”) optimism, and the reported interval captures only test-set sampling noise, not train/validation-split or random-seed variability. Cross-validated or repeated-seed evaluation would tighten these estimates and is left to future work. Exact winning configurations for the reported deep models are not recoverable from the saved artifacts; Table 2 therefore documents the common protocol and recorded candidate-run settings.

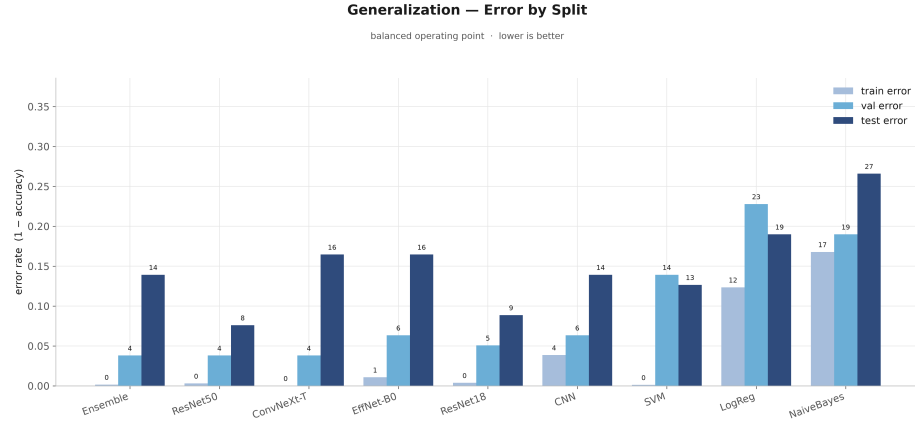
The neural models also depend on training-time augmentation to increase effective sample diversity. Augmentation makes the models less sensitive to controlled transformations, but it does not replace additional real images containing natural occlusion, motion blur, dense crowding, and non-standard bus or leguna structures. These factors can make unsafe/risky positive cases difficult to distinguish from visually cluttered safe/normal scenes. In addition, the split code stratifies by vehicle type and class but does not group by scene, event, or near-duplicate identity; residual dependence across partitions therefore cannot be ruled out from the repository evidence.

The current system is limited to image-level binary classification. It does not localize hanging passengers, estimate crowd density, track passengers across frames, or evaluate performance on large-scale real-time video streams. Roadside transport images may also include identifiable people, license plates, and location cues, so deployment would require anonymization and data-governance safeguards. The method should therefore be interpreted as a baseline for visual safety classification rather than an operational monitoring system.

Table 5: Train, Validation, and Test Generalization

Model	Train Acc.	Train Err.	Val Acc.	Val Err.	Test Acc.	Test Err.	Test R
Ensemble	0.999	0.001	0.962	0.038	0.861	0.139	0.947
ResNet50	0.997	0.003	0.962	0.038	0.924	0.076	0.921
ConvNeXt-Tiny	1.000	0.000	0.962	0.038	0.835	0.165	0.947
EfficientNet-B0	0.989	0.011	0.937	0.063	0.835	0.165	0.974
ResNet18	0.996	0.004	0.949	0.051	0.911	0.089	0.921
CNN	0.962	0.038	0.937	0.063	0.861	0.139	0.868
SVM (RBF)	0.999	0.001	0.861	0.139	0.873	0.127	0.895
Logistic Regression	0.877	0.123	0.772	0.228	0.810	0.190	0.816
Naive Bayes	0.832	0.168	0.810	0.190	0.734	0.266	0.737

(a) Train, validation, and test error.



(b) Train-to-test generalization gap.

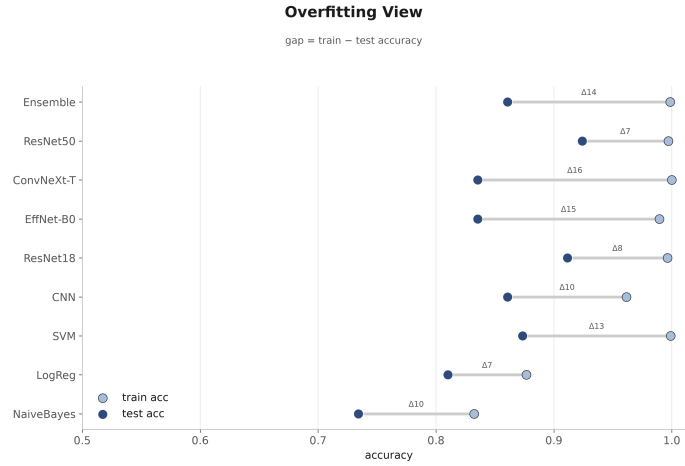


Figure 11: Generalization behavior of all nine models after retraining.

8 Conclusion and Future Work

This paper developed a computer-vision and machine-learning pipeline for detecting unsafe hanging travel in Dhaka public transport. The task was formulated as binary image classification over safe/normal and unsafe/risky passenger conditions. A localized bus and leguna dataset was prepared through cleaning, annotation, preprocessing, and augmentation, with HOG features used for classical models and RGB inputs used for CNN-based classification.

The updated result set compares classical models (Logistic Regression, Gaussian Naive Bayes, and SVM with an RBF kernel) with deep models (a CNN trained from scratch, ResNet18, EfficientNet-B0, ResNet50, ConvNeXt-Tiny, and a ResNet50–ConvNeXt ensemble). On the untouched test set, the deployed ensemble reaches a recall-oriented balanced result with 86.1% accuracy, 94.7% unsafe recall, and 0.970 ROC-AUC, while ResNet50 gives the highest single-model accuracy at 92.4%. Because the retrained data is near-balanced, the balanced, high-recall, and max-recall thresholds coincide at the same 86.1%-accuracy and 94.7%-recall operating point rather than trading accuracy for a separate perfect-recall mode.

As a practical outcome of this work, we developed an online platform that classifies public-transport images as safe/normal or unsafe/risky. The project resources are publicly accessible through the source-code repository, Mendeley Data, and the deployed web platform.

Future work should focus on collecting a larger localized dataset, improving recall for unsafe hanging travel, evaluating the system on real-time video streams, and designing privacy-aware deployment procedures. Detection-based models such as YOLO [6] should also be tested to localize hanging passengers rather than only classify whole images. License-plate recognition could be incorporated in future work to identify vehicles after an unsafe condition has been detected.

References

- [1] E. Dilek and M. Dener, “Computer Vision Applications in Intelligent Transportation Systems: A Survey,” *Sensors*, vol. 23, no. 6, 2938, 2023.
<https://www.mdpi.com/1424-8220/23/6/2938>
- [2] Y.-W. Hsu, T.-Y. Wang, and J.-W. Perng, “Passenger flow counting in buses based on deep learning using surveillance video,” *Optik*, vol. 202, 163675, 2020.
<https://www.sciencedirect.com/science/article/abs/pii/S0030402619315736>
- [3] S. Barua, M. Seraj, and D. Alam, “Opportunities and Operational Difficulties of Introducing Bus Rapid Transit (BRT) in Dhaka,” in *Urban Public Transportation Systems 2013*, American Society of Civil Engineers, 2013,

pp. 168-178.

doi: 10.1061/9780784413210.016

- [4] Z. Li and D. A. Hensher, "Crowding in Public Transport: A Review of Objective and Subjective Measures," *Journal of Public Transportation*, vol. 16, no. 2, pp. 107-134, 2013.
doi: 10.5038/2375-0901.16.2.6
- [5] L. M. Wastupranata, S. G. Kong, and L. Wang, "Deep Learning for Abnormal Human Behavior Detection in Surveillance Videos—A Survey," *Electronics*, vol. 13, no. 13, 2579, 2024.
<https://www.mdpi.com/2079-9292/13/13/2579>
- [6] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi, "You Only Look Once: Unified, Real-Time Object Detection," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 779-788.
arXiv:1506.02640
- [7] W. Xu, Y. Zhao, X. Du, H. Ji, and L. Xing, "A Study on Bus Passenger Boarding and Alighting Detection and Recognition Based on Video Images and YOLO Algorithm," *Sensors*, vol. 26, no. 5, 1418, 2026.
doi: 10.3390/s26051418
- [8] C.-H. Tseng and H.-Y. Lin, "A Vision-Based System for Abnormal Behavior Detection and Recognition of Bus Passengers," in *2022 IEEE 25th International Conference on Intelligent Transportation Systems (ITSC)*, 2022, pp. 1-6.
doi: 10.1109/ITSC55140.2022.9921801
- [9] H.-Y. Lin and C.-H. Tseng, "Abnormal Activity Detection and Classification of Bus Passengers With In-Vehicle Image Sensing," *IEEE Access*, vol. 12, pp. 23057-23065, 2024.
doi: 10.1109/ACCESS.2024.3365138
- [10] S. Liu, Y. Bi, Q. Li, Y. Ren, H. Ji, and L. Wang, "A deep learning based detection algorithm for anomalous behavior and anomalous item on buses," *Scientific Reports*, vol. 15, art. 2163, 2025.
doi: 10.1038/s41598-025-85962-8
- [11] H. Ramirez, S. Seriani, V. Aprigliano, A. Peña, B. Arredondo, I. Bastias, and G. Farias, "Posture Detection of Passengers' Movement When Boarding and Alighting an Urban Bus: A Pilot Study in Valparaíso, Chile," *Applied Sciences*, vol. 15, no. 10, 5367, 2025.
doi: 10.3390/app15105367
- [12] C. N. E. Anagnostopoulos, I. E. Anagnostopoulos, I. D. Psoroulas, V. Loumos, and E. Kayafas, "License Plate Recognition From Still Images and Video Sequences: A Survey," *IEEE Transactions on Intelligent Transportation Systems*, vol. 9, no. 3, pp. 377-391, 2008.
doi: 10.1109/TITS.2008.922938

- [13] S. M. Silva and C. R. Jung, "License Plate Detection and Recognition in Unconstrained Scenarios," in *European Conference on Computer Vision (ECCV)*, 2018, pp. 593-609.
https://link.springer.com/chapter/10.1007/978-3-030-01258-8_36
- [14] S. Du, M. Ibrahim, M. Shehata, and W. Badawy, "Automatic License Plate Recognition (ALPR): A State-of-the-Art Review," *IEEE Transactions on Circuits and Systems for Video Technology*, vol. 23, no. 2, pp. 311-325, 2013.
doi: 10.1109/TCSVT.2012.2203741
- [15] N. Afrin, M. M. Hasan, M. F. E. Safin, K. R. Amin, M. Z. Haque, F. Ahmed, and M. T. R. Shawon, "Bengali License Plate Recognition: Unveiling Clarity with CNN and GFP-GAN," in *26th International Conference on Computer and Information Technology (ICCIT)*, Cox's Bazar, Bangladesh, 2023, pp. 1-6.
arXiv:2312.10701
- [16] S. Tabassum, S. Ullah, N. H. Al-nur, and S. Shatabda, "Poribohon-BD: Bangladeshi local vehicle image dataset with annotation for classification," *Data in Brief*, vol. 33, 106465, 2020.
doi: 10.1016/j.dib.2020.106465
- [17] N. Indhumathi, S. Maheswaran, M. Shunmuganantha, P. A. Vikash, and R. S. Vimal Raj, "Bus Foot Board Control Using IoT," in *2024 15th International Conference on Computing Communication and Networking Technologies (ICCCNT)*, 2024, pp. 1-6.
doi: 10.1109/ICCCNT61001.2024.10725076
- [18] C. Shorten and T. M. Khoshgoftaar, "A survey on Image Data Augmentation for Deep Learning," *Journal of Big Data*, vol. 6, art. 60, 2019.
<https://link.springer.com/article/10.1186/s40537-019-0197-0>
- [19] N. Dalal and B. Triggs, "Histograms of Oriented Gradients for Human Detection," in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2005, pp. 886-893.
<https://lear.inrialpes.fr/people/triggs/pubs/Dalal-cvpr05.pdf>
- [20] S. van der Walt, J. L. Schönberger, J. Núñez-Iglesias, F. Boulogne, J. D. Warner, N. Yager, E. Gouillart, and T. Yu, "scikit-image: image processing in Python," *PeerJ*, vol. 2, e453, 2014.
<https://peerj.com/articles/453/>
- [21] C. Cortes and V. Vapnik, "Support-Vector Networks," *Machine Learning*, vol. 20, no. 3, pp. 273-297, 1995.
<https://link.springer.com/article/10.1007/BF00994018>

- [22] A. Krizhevsky, I. Sutskever, and G. E. Hinton, “ImageNet Classification with Deep Convolutional Neural Networks,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 25, 2012, pp. 1097-1105.
<https://papers.nips.cc/paper/4824-imagenet-classification-with-deep-convolutional-neural-networks>
- [23] K. He, X. Zhang, S. Ren, and J. Sun, “Deep Residual Learning for Image Recognition,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 770-778.
arXiv:1512.03385
- [24] M. Tan and Q. V. Le, “EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks,” in *International Conference on Machine Learning (ICML)*, 2019, pp. 6105-6114.
arXiv:1905.11946
- [25] Z. Liu, H. Mao, C.-Y. Wu, C. Feichtenhofer, T. Darrell, and S. Xie, “A ConvNet for the 2020s,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022, pp. 11966-11976.
arXiv:2201.03545
- [26] S. Ioffe and C. Szegedy, “Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift,” in *International Conference on Machine Learning (ICML)*, 2015, pp. 448-456.
arXiv:1502.03167
- [27] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, “Dropout: A Simple Way to Prevent Neural Networks from Overfitting,” *Journal of Machine Learning Research*, vol. 15, pp. 1929-1958, 2014.
<https://jmlr.org/papers/v15/srivastava14a.html>
- [28] D. Chicco and G. Jurman, “The advantages of the Matthews correlation coefficient (MCC) over F1 score and accuracy in binary classification evaluation,” *BMC Genomics*, vol. 21, art. 6, 2020.
doi: 10.1186/s12864-019-6413-7
- [29] F. Pedregosa et al., “Scikit-learn: Machine Learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825-2830, 2011.
<https://jmlr.org/papers/v12/pedregosa11a.html>
- [30] A. Paszke et al., “PyTorch: An Imperative Style, High-Performance Deep Learning Library,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 32, 2019, pp. 8024-8035.
<https://papers.nips.cc/paper/9015-pytorch-an-imperative-style-high-performance-deep-learning-library>
- [31] D. P. Kingma and J. Ba, “Adam: A Method for Stochastic Optimization,” in *International Conference on Learning Representations (ICLR)*, 2015.
arXiv:1412.6980